university of groningen

faculty of science and engineering

Training a CNN Classifier on the CIFAR-10 Dataset

Marcus Persson (S5343798),
Emily Heugen (S5587042),
Richard Harnisch (S5238366)

Contents

1 Introduction

Convolutional Neural Networks (CNNs) have significantly transformed the landscape of computer vision, providing an automated approach for feature extraction and hierarchical learning of spatial patterns within images. Unlike traditional machine learning techniques that rely on handcrafted features, CNNs utilize convolutional layers to detect fundamental low-level features, such as edges and textures. These features are progressively combined to form more complex high-level representations, enabling CNNs to excel in tasks such as image classification, object detection, and semantic segmentation. This advancement has solidified CNNs as the cornerstone of modern deep learning in computer vision [1].

One widely used benchmark for evaluating CNN models is the CIFAR-10 dataset [2], which comprises 60,000 color images categorized into 10 distinct classes. With a resolution of 32x32 pixels, the relatively small image size and diversity in object representation make CIFAR-10 a challenging yet ideal platform for testing and improving CNN architectures. The dataset's standardization allows for consistent performance comparisons across various models, making it a critical tool for advancing research in the field.

This project focuses on the implementation and training of CNN models on the CIFAR-10 dataset, exploring how different model configurations and training strategies influence performance. Initially, a simple CNN model will be implemented using TensorFlow, incorporating preprocessing techniques like data normalization and augmentation, along with hyperparameter optimization. The model will be trained, and its performance evaluated by analyzing classification errors and identifying opportunities for improvement.

While the project begins with a simple CNN architecture, we also recognize the potential of more advanced models. For instance, ResNet18 [3], a deep residual network, has been shown to outperform traditional CNNs due to its use of skip connections that help mitigate the vanishing gradient problem and facilitate the training of deeper networks. By comparing the performance of the baseline CNN to that of a fine-tuned ResNet18 model on the same dataset, we aim to better understand the trade-offs between model complexity, training time, and accuracy.

Throughout all this, the study will aim to answer the following question: "How can a simple convolutional neural network (CNN) classifier be implemented, trained, and evaluated on the CIFAR-10 dataset, and how does its performance compare to that of a fine-tuned pre-trained model such as ResNet18?"

2 Problem Definition and Algorithm

2.1 Problem Definition

The problem addressed is image classification using the CIFAR-10 dataset, which consists of 60,000 images across 10 object categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The goal is to develop a machine learning model that, given an input image x, correctly predicts the corresponding class label y, where y is an element of the set of 10 predefined categories.
Formally, the task can be defined as learning a function $f : X \to Y$ that maps an input image $x \in X$ to a label $y \in Y$, where X is the space of all images in CIFAR-10 and $Y = \{0, 1, 2, \ldots, 9\}$ represents the set of class labels. The model is trained on a labeled dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$, where each image x_i is associated with a true class label y_i, and evaluated based on its ability to generalize to unseen test samples.

2.2 Overview of the Algorithms Used

Two deep learning models are employed to address the classification task: a custom convolutional neural network (CNN) and a fine-tuned ResNet18 model using transfer learning. Both models are trained using supervised learning, where the goal is to minimize the categorical cross-entropy loss function:

$$L = -\sum_{i=1}^{N}\sum_{j=1}^{C} y_{i,j}\log(\hat{y}_{i,j}) \tag{1}$$

where N is the number of training samples, C is the number of classes, $y_{i,j}$ is the true label (one-hot encoded), and $\hat{y}_{i,j}$ is the predicted probability for class j. The training process involves updating the model parameters using backpropagation and the Adam optimizer.

2.3 Custom CNN Algorithm

The custom CNN consists of three convolutional layers, each followed by batch normalization, ReLU activation, and max pooling. The feature maps are flattened and passed through two fully connected layers, with dropout regularization applied before the final softmax output.

2.4 ResNet18 Transfer Learning Algorithm

The second approach utilizes ResNet18, a pre-trained deep residual network, to extract high-level image features. The base layers of ResNet18 remain frozen to retain learned feature representations, while a new classification head is trained for CIFAR-10. The classification head consists of a global average pooling layer, a dense layer (128 neurons, ReLU activation), and a final softmax layer. The training follows the same supervised learning process, but only the newly added layers are updated.

2.5 Example Walkthrough

To illustrate the difference in processing between the two models, consider an example input image of a ”dog” (Class 5). In the custom CNN:

1. The input image passes through three convolutional layers, each progressively extracting low-to-high level features such as edges, textures, and shapes.
2. The flattened features are fed into fully connected layers, where the model learns to associate them with a specific class.
3. The final softmax layer outputs a probability distribution across the 10 classes, with the highest probability ideally assigned to Class 5 (dog).

In contrast, in the ResNet18 model:

1. The input passes through the pre-trained convolutional layers, leveraging features learned from ImageNet (e.g., detecting fur patterns, eyes, and limbs).
2. The classification head processes the extracted features, mapping them to the 10 CIFAR-10 classes.
3. The softmax layer generates class probabilities, and ideally, the highest probability corresponds to Class 5 (dog).

2.6 Summary of Algorithm Choices

The custom CNN is optimized specifically for CIFAR-10 with a lightweight architecture suited for small-scale datasets, while ResNet18 leverages knowledge transfer from ImageNet but may suffer from feature mismatches. This study compares these models to evaluate the trade-offs between domain-specific architecture design and transfer learning for low-resolution image classification.

3 Methods

3.1 Evaluation Criteria and Hypotheses

The performance of the model on the CIFAR-10 dataset is evaluated using quantitative metrics such as classification accuracy, loss, precision, recall, and F1-score (computed on a per-class basis). Our primary hypotheses are twofold. First, we posit that a custom-built convolutional neural network (CNN) enhanced by data augmentation can achieve competitive performance on CIFAR-10. Second, we hypothesize that leveraging a pre-trained ResNet18 model through transfer learning—with the base layers frozen and a new classifier head added—can improve or at least match the performance of the custom CNN. Both models are monitored by tracking training and validation metrics over 20 epochs, and their effectiveness is evaluated using confusion matrices and derived statistical measures.

3.2 Data and Preprocessing

The experiments are conducted on the CIFAR-10 dataset, which comprises 60,000 images (50,000 for training and 10,000 for testing) across 10 classes. All images are normalized so that pixel values lie in the range [0, 1], a step essential for enhancing training stability. To mitigate overfitting and improve model generalizability, a data augmentation pipeline is implemented using TensorFlow; this pipeline applies random horizontal flips, rotations (up to 10%), and zoom operations (up to 10%) during training. The training data is shuffled, batched with a batch size of 32, and preloaded using TensorFlow's prefetching capabilities to ensure efficient data processing during the training phase.

3.3 Custom CNN Model Architecture and Training

For the custom CNN model, the architecture consists of three convolutional blocks with progressively increasing filter sizes (32, 64, and 128). Each block incorporates batch normalization and max pooling. After flattening the feature maps, a dropout layer (set at 50%) is introduced to reduce overfitting before the data passes through two fully connected layers, with the final layer using a softmax activation function to output class probabilities. In this model, the independent variables include architectural design choices (e.g., the number of convolutional layers, filter sizes, dropout rate) and the use of data augmentation, while the dependent variables are the training and validation accuracy and loss. The model is trained for 20 epochs using the Adam optimizer with a learning rate of 0.001, and performance data is collected after each epoch for subsequent analysis.

3.4 Fine-Tuned ResNet18 Model Architecture and Training

The transfer learning approach employs a pre-trained ResNet18 model as a fixed feature extractor by freezing its base layers to retain the learned ImageNet representations. A new classification head, consisting of a global average pooling layer followed by a dense layer (128 units) and a final softmax output layer for the 10 classes, is appended to adapt the model to the CIFAR-10 classification task. The independent variables for this approach include the pre-trained weights, the configuration of the new classification head, and the same augmentation strategy and learning rate used in the custom CNN experiment. Like the custom model, the ResNet18-based model is trained for 20 epochs under identical data handling and batching conditions, and its performance is evaluated based on training/-validation metrics and confusion matrix-derived statistics.

3.5 Data Analysis and Performance Metrics

Performance is analyzed by plotting training and validation accuracy and loss curves over the epochs, which provide visual insight into the learning dynamics of each model. Detailed confusion matrices are generated to compute per-class precision, recall, and F1-score, offering a granular view of model performance across different classes. Comparative analysis reveals that the custom CNN achieves a test accuracy of approximately 70.7% with a test loss of 0.8668, while the fine-tuned ResNet18 model attains a lower test accuracy (around 27.8%) with a higher test loss (1.9298). These significant differences suggest that the custom CNN, which is designed specifically for the CIFAR-10 dataset and benefits from tailored data augmentation, is better suited to this task under the current experimental conditions. In contrast, the ResNet18 model may have underperformed due to the mismatch between the pre-trained ImageNet features and the lower-resolution CIFAR-10 images, as well as the limitations inherent in freezing its base layers.

4 Results

4.1 Custom CNN Model

The custom CNN achieved a test accuracy of 70.72%, with a final test loss of 0.8668. The precision, recall, and F1-score (Table 1) indicate that the model was able to correctly classify most images, although certain classes exhibited lower recall, meaning some images were frequently misclassified. Figure 1 displays a confusion matrix indicating each class' classification strength.

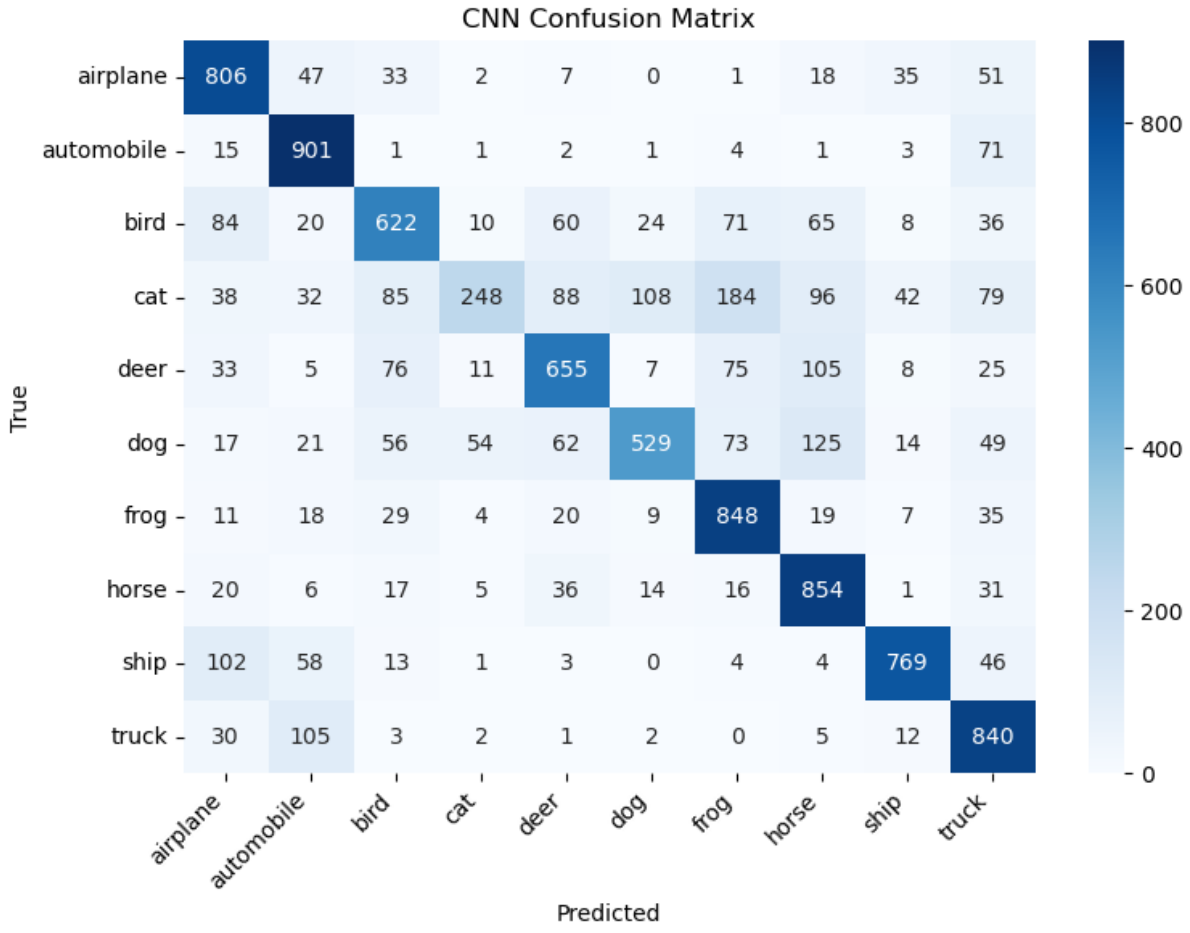

Figure 1: Confusion Matrix for the custom CNN Model

Cats were especially often misclassified, for example, as horses, frogs, dogs, or deer. This is likely because of the visual similarity and shared context in which they are photographed. The class with the highest recall was "automobile". This also makes sense considering the context in which pictures are taken, no other class would likely be on a largely gray background other than "truck", which is the most likely to be confused with automobile.

Class	airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck
Precision	0.6972	0.7428	0.6652	0.7337	0.7013	0.7622	0.6646	0.6610	0.8554	0.6651
Recall	0.8060	0.9010	0.6220	0.2480	0.6550	0.5290	0.8480	0.8540	0.7690	0.8400
F1-Score	0.7477	0.8143	0.6429	0.3707	0.6774	0.6246	0.7452	0.7452	0.8099	0.7424

Table 1: Selected metrics for our custom CNN

4.2 ResNet18 Transfer Learning

The fine-tuned ResNet18 model performed significantly worse, achieving a test accuracy of 27.79% with a much higher test loss of 1.9298. This suggests that ResNet18 struggled to generalize to CIFAR-10 despite leveraging pre-trained weights from ImageNet. Figure 2 displays a confusion matrix indicating each class' classification strength.

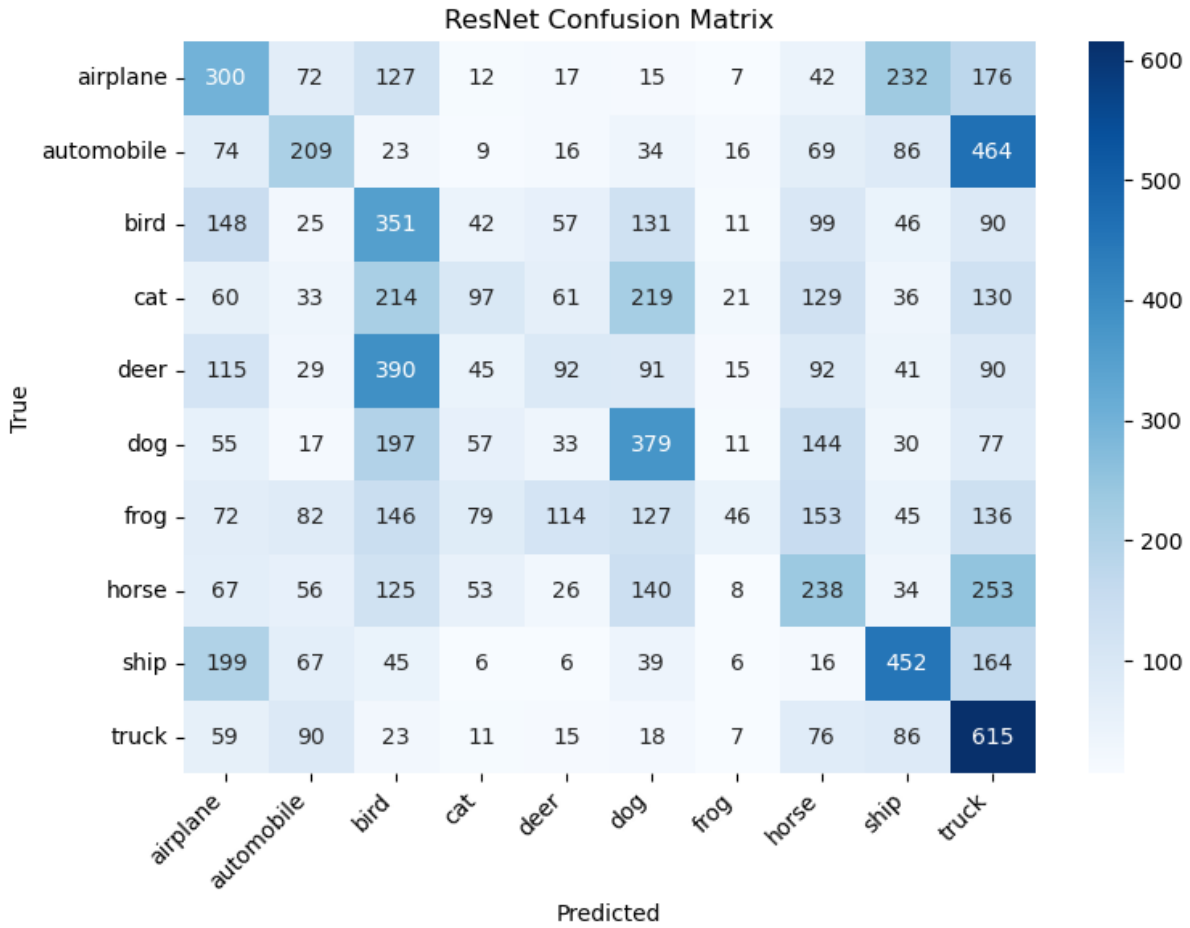

Figure 2: Confusion Matrix for the ResNet18 Model

Frogs were almost never predicted at all, with only 4.6% recall. This indicates that the model almost failed entirely to recognize frogs. Table 2 reports on the poor precision, recall, and F1-score metrics from the ResNet18 model.

Class	airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck
Precision	0.2611	0.3074	0.2139	0.2360	0.2105	0.3177	0.3108	0.2250	0.4154	0.2802
Recall	0.3000	0.2090	0.3510	0.0970	0.0920	0.3790	0.0460	0.2380	0.4520	0.6150
F1-Score	0.2792	0.2488	0.2658	0.1375	0.1280	0.3456	0.0801	0.2313	0.4330	0.3850

Table 2: Selected metrics for the finetuned ResNet

Figure 3: Test accuracy and loss for the CNN and ResNet models

4.3 Comparison

Overall, we can say that the custom CNN performed significantly better than the finetuned ResNet18. This is most evident in figure 4 which compares the train and test validation accuracies between the two models. This is likely because the CNN is suited more specifically to this dataset. The CNN is tailored to the 32x32 image size, and the model architecture employs data augmentation techniques such as flipping and rotating images. This helped it learn features more robustly. Additionally, the large size (224x224px) that ResNet was originally trained on likely means that the low-level features it was trained to recognize are not present in the CIFAR dataset, as there are simply not enough pixels.

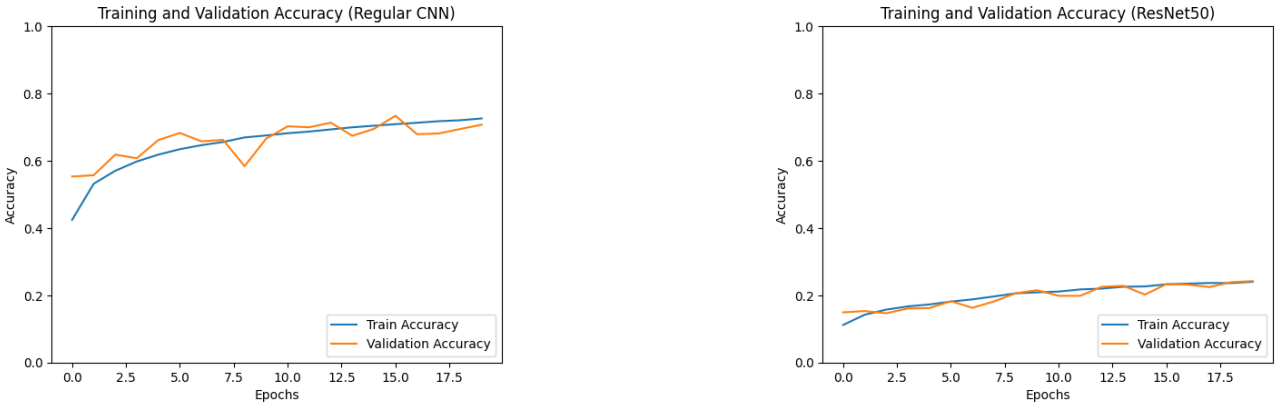

Figure 4: Train and Test/Validation accuracy during training

The training and test loss curves of figure 4 do not indicate overfitting. Both models show initially faster progress that slows down as the models reach the end of the training. Since we did not implement early stopping or other techniques that could cause data leakage, no validation set is needed. We are also using a well-respected and trusted dataset that might otherwise introduce data leakage. Figure 5 demonstrates the comparisons between precision, recall, and F-1 score between the two models.

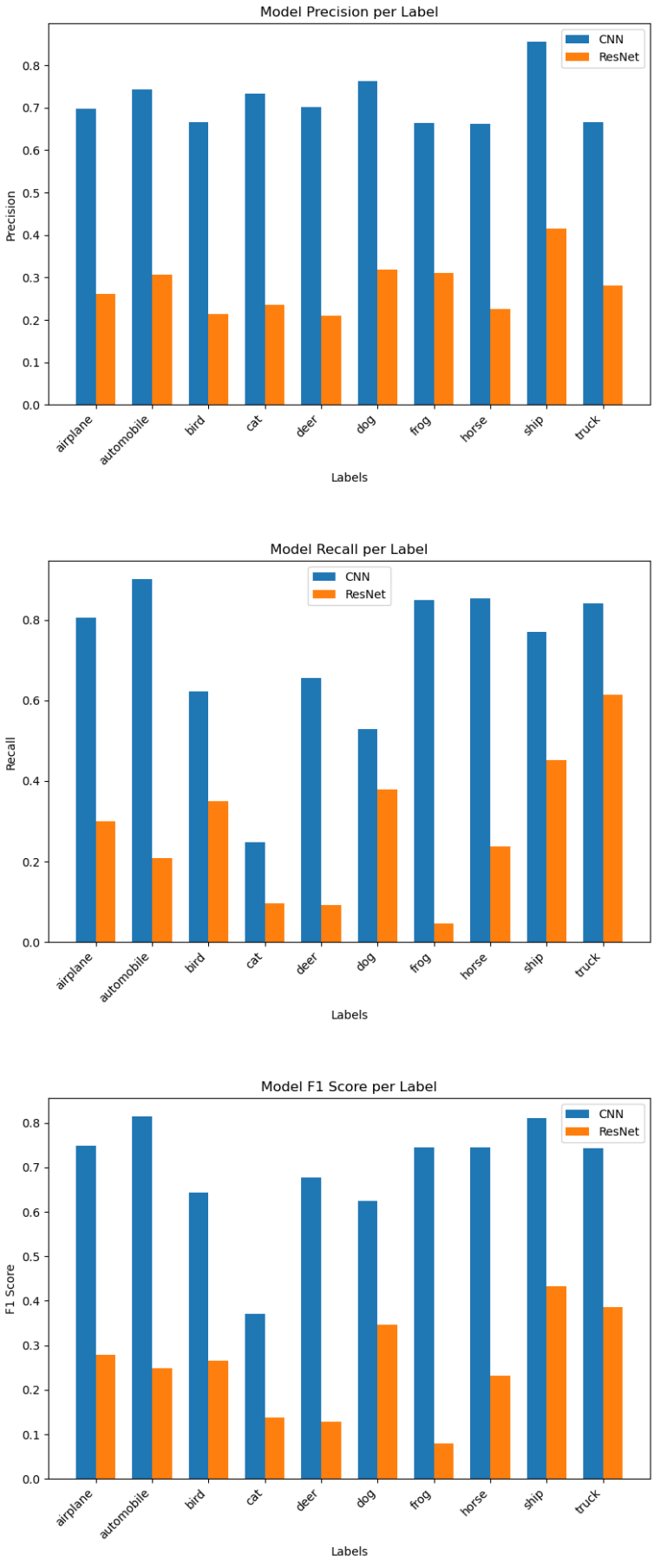

Figure 5: Precision, Recall, and F1-Score per data class by model

5 Conclusion

This study explored the implementation and training of convolutional neural networks (CNNs) for image classification using the CIFAR-10 dataset. Two approaches were compared: a custom CNN model designed specifically for CIFAR-10 and a fine-tuned ResNet18 model leveraging transfer learning from ImageNet. The results indicate that a carefully designed CNN tailored to the dataset can significantly outperform a deep, pre-trained model when dataset characteristics differ from the original training domain.

The custom CNN achieved a test accuracy of 70.72%, demonstrating strong performance given its relatively simple architecture. Key factors contributing to this success included data augmentation, dropout regularization, and careful selection of hyperparameters. In contrast, the fine-tuned ResNet18 model underperformed, achieving a much lower test accuracy of 27.79%, suggesting that knowledge transfer from large-scale datasets like ImageNet does not always translate effectively to small, low-resolution datasets like CIFAR-10.

These findings highlight the importance of dataset-specific model design. While transfer learning can be a powerful technique, its effectiveness depends on how closely the source and target datasets align. In cases where dataset characteristics diverge significantly, as seen with ResNet18 on CIFAR-10, a well-optimized CNN tailored to the problem domain may yield better results.

Overall, this study reinforces the idea that **bigger is not always better** when it comes to deep learning. Instead, selecting the right model for the dataset and optimizing it appropriately remains key to achieving strong performance in computer vision tasks.

5.1 Summary of Main Contributions

All group members collaborated equally on all parts of the project and report.

5.2 Future Work

Future research can explore enhancing transfer learning by selectively fine-tuning deeper layers of ResNet18 instead of freezing the entire feature extractor. This approach may allow the model to better adapt to CIFAR-10's lower-resolution images while still leveraging the benefits of pre-trained ImageNet representations. Additionally, investigating alternative architectures such as MobileNet or EfficientNet could provide insights into optimizing performance on small-scale datasets, as these models are designed to be lightweight while maintaining strong classification accuracy.

Another promising direction is the implementation of advanced data augmentation techniques, such as CutMix or MixUp, which could improve generalization by exposing the model to more diverse training samples. These methods have been shown to enhance robustness by blending different images and labels during training, thereby reducing overfitting and increasing classification performance.

Hyperparameter optimization is another crucial avenue for future improvement. Instead of manually tuning parameters, automated techniques such as Bayesian optimization or genetic algorithms could be employed to systematically search for optimal learning rates, batch sizes, and architectural choices. This would ensure that the models are trained with the most effective configurations, potentially leading to higher accuracy and better generalization.

Finally, expanding the dataset or applying domain adaptation techniques could further test the generalizability of these models. Since CIFAR-10 is a relatively small dataset, experiments on larger and more diverse datasets, such as CIFAR-100 or Tiny ImageNet, could provide a more comprehensive

evaluation of the proposed architectures. By exploring these directions, future work can refine CNN-based approaches for small-scale image classification and push the boundaries of what is achievable with deep learning on constrained datasets.

Bibliography

[1] M. Huang, “Image recognition with cnns: improving accuracy and efficiency,” Feb 2023.

[2] A. Krizhevsky, “Cifar-10 and cifar-100 datasets,” *www.cs.toronto.edu*, 2013.

[3] Mathworks, “Resnet18,” *Mathworks.com*, 2020.

A Appendix

A.1 Jupyter Notebook File

This is an extension to the report, which documents the processes of pre-processing the dataset and training and evaluating the CNN and ResNet18 models. It is attached alongside this report and contains the codework behind the report.